TECHNICAL DOCUMENTATION

AccessLens Documentation

How AccessLens works, how to run it, and the real issues found while building it.

Author Ubaid Raza Ansari
GRC and IAM Analyst MSc Cyber Security NCSC Certified

Live demo: https://ubaidraza5.github.io/accesslens

Source code: https://github.com/ubaidraza5/accesslens

What this adds up to

AccessLens

AccessLens reviews an identity and access export and tells you, in plain language, which accounts need attention and why. The same review also produces a full technical breakdown for a security or audit analyst, built from the exact same analysis, so the two views can never disagree with each other.

Live demo: https://ubaidraza5.github.io/accesslens Source code: https://github.com/ubaidraza5/accesslens

What it looks for

AccessLens checks an access export against four common review areas.

Dormant and orphaned access looks for accounts that have not been used in a long time, and for access that is still active even though the employee record shows they have already left the company.

Excess privilege looks for a person who holds a much higher level of access than everyone else on their team, and for a system where too many people hold admin rights at once.

Separation of duties looks for a person who holds two permissions that are supposed to be kept apart, for example being able to both create and approve the same payment.

Ownership and review checks that every account has a manager listed, and flags access that has not been reviewed in over a year.

Every issue found is called a finding. Each finding carries a plain sentence explaining what is wrong, the evidence behind it, a recommended next step, and a severity from Info up to Critical. Findings are combined into a single score out of 100 for the whole company, and a score for each account.

Two ways to read the same report

Simple mode is written for anyone in the business. It names the accounts that need attention first and explains what is wrong in a sentence, with no jargon and no codes.

Analyst mode shows the full technical picture, every finding, the evidence behind it, and the severity breakdown, grouped by account and by the system wide issues that are not tied to a single person.

Both modes are generated from one shared analysis pass, there is no second pass or separate logic that could tell a different story.

Try it without installing anything

Open the live demo, load one of the two sample companies, and press Run review. Nothing you paste or upload is ever sent anywhere, the whole analysis runs in your own browser using the JavaScript port of the same engine described below.

Running it from the command line

AccessLens needs only the Python standard library, there is nothing to install for normal use.

```
python main.py your_access_export.csv
```

This prints the plain language summary. For the full technical view:

```
python main.py your_access_export.csv --mode analyst
```

To also save the full report as JSON:

```
python main.py your_access_export.csv --json report.json
```

The command exits with a nonzero status when the overall risk comes back High or Critical, which makes it easy to use as a gate in an automated pipeline.

The input file

AccessLens reads a CSV export with these columns.

COLUMN	WHAT GOES IN IT
name	The person's full name
email	Their email address, used as the unique account identifier
department	The team or department they belong to
system	The system or application this row grants access to
permission	The permission or role granted, for example Admin or Standard User
date_granted	The date the access was granted, YYYY MM DD
date_last_used	The date the access was last used, left blank if never used
employee_status	Active or Terminated
manager	The manager who owns or approved this access

Each row is one person's access to one system. A person with access to three systems appears as three rows. This matches the shape of a typical export from Active Directory, Okta, Azure AD, or a cloud provider's console.

A sample export with a full set of realistic, deliberately planted issues lives at `tests/fixtures/sample_company_access.csv`, and a clean export with no issues at all lives at `tests/fixtures/clean_company_access.csv`. Both use entirely fictional people and a fictional company, no real company data is included anywhere in this project.

Project layout

```
access_parser.py       reads the CSV and builds AccessRecord objects
finding.py              the shared Finding and Severity types
dormant_analyzer.py     dormant and orphaned access checks
privilege_analyzer.py   excess privilege and admin concentration checks
sod_analyzer.py         separation of duties conflict checks
ownership_analyzer.py   missing manager and overdue review checks
risk_scorer.py          combines findings into account and company scores
report_generator.py     renders the simple, analyst, and JSON reports
main.py                 the command line entry point
web/index.html          the browser based demo, a JavaScript port of the engine
scripts/verify_web_demo.js   checks the web demo agrees with the Python engine
tests/                  the pytest suite and sample fixtures
docs/                   the walkthrough and supporting documentation
```

Testing

```
pip install -r requirements-dev.txt
pytest
```

The web demo has its own check, which extracts the exact script shipped in `web/index.html` and runs it against the same fixture files the Python tests use, confirming both engines produce the same score, level, and finding

counts.

```
node scripts/verify_web_demo.js
```

Both checks run automatically in CI on every change, and the browser demo only deploys once they both pass.

Development notes

`DEVELOPMENT_NOTES.md` records the real problems found while building this project, and how each one was fixed, from a scoring edge case to a GitHub Pages setting that needed changing before the live demo would load.

Security and privacy

See `SECURITY.md`. In short, nothing pasted or uploaded to the browser demo ever leaves the page, and the CLI only ever reads the file you point it at.

License

MIT, see `LICENSE`.

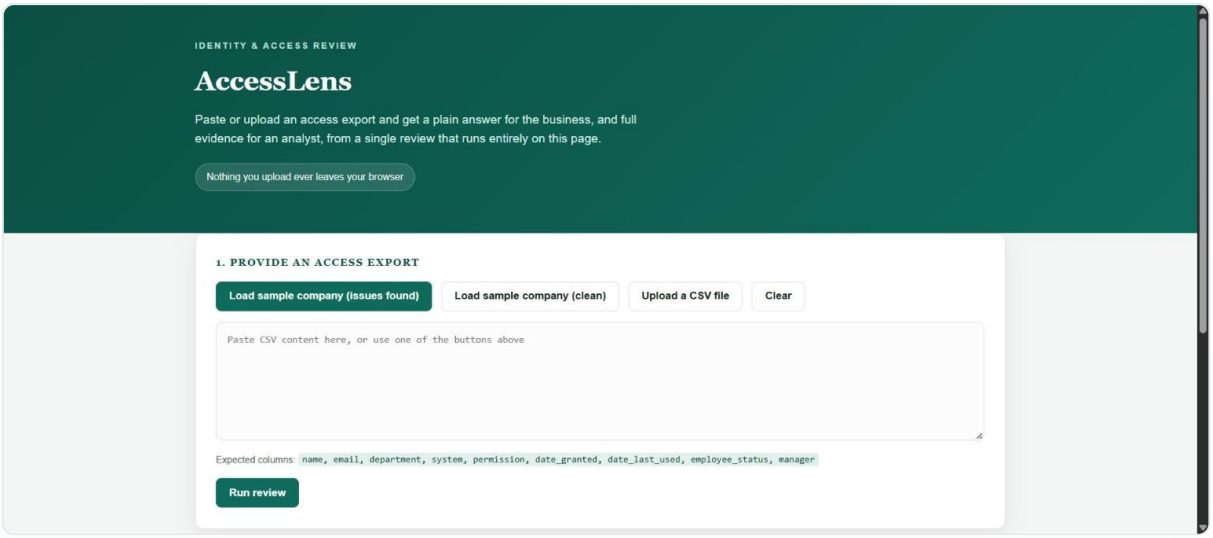
Author

Built by Ubaid Raza Ansari, GRC and IAM Analyst, MSc Cyber Security, NCSC Certified.

Screenshots, the project in action

These are real screenshots of the live demo and the GitHub repository, taken directly from the working project.

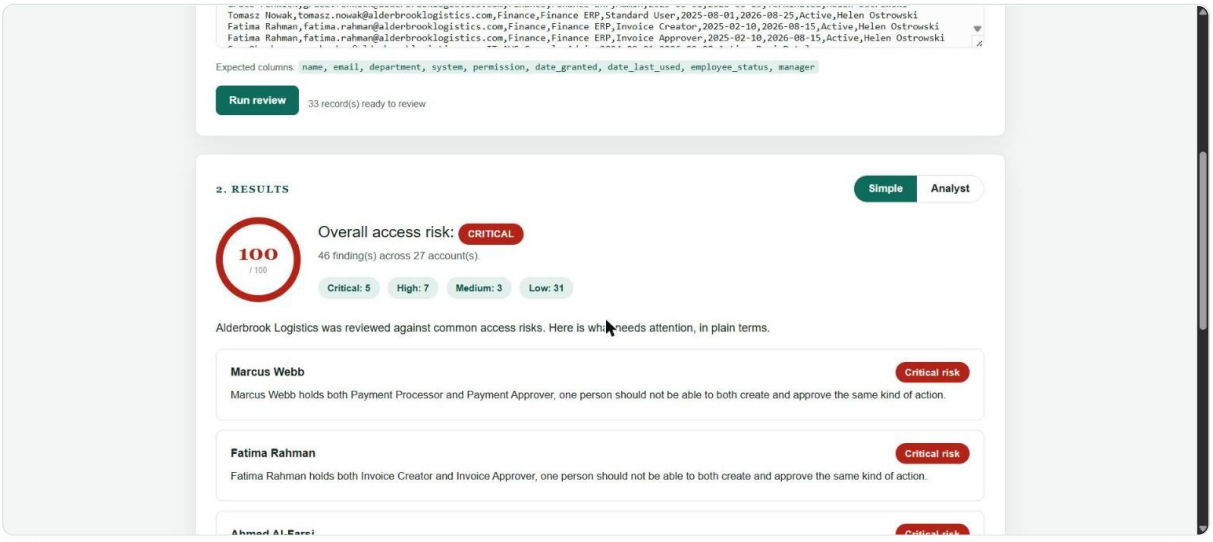
The homepage



The AccessLens homepage

The page explains itself in a sentence, paste or upload an access export and the check runs entirely on your own screen. Nothing you paste or upload is ever sent anywhere.

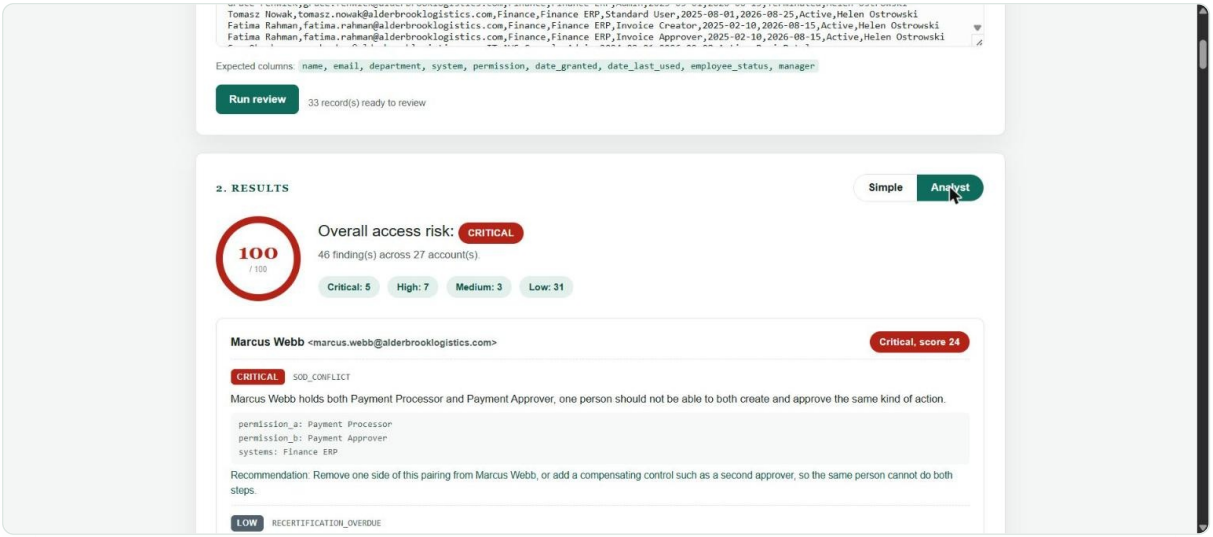
The simple, plain language report



The simple report on a company with planted issues

Alderbrook Logistics is a fictional company export with a full set of deliberately planted issues. The simple view names the accounts that need attention first and explains what is wrong in one sentence, written for anyone in the business, not just a security team.

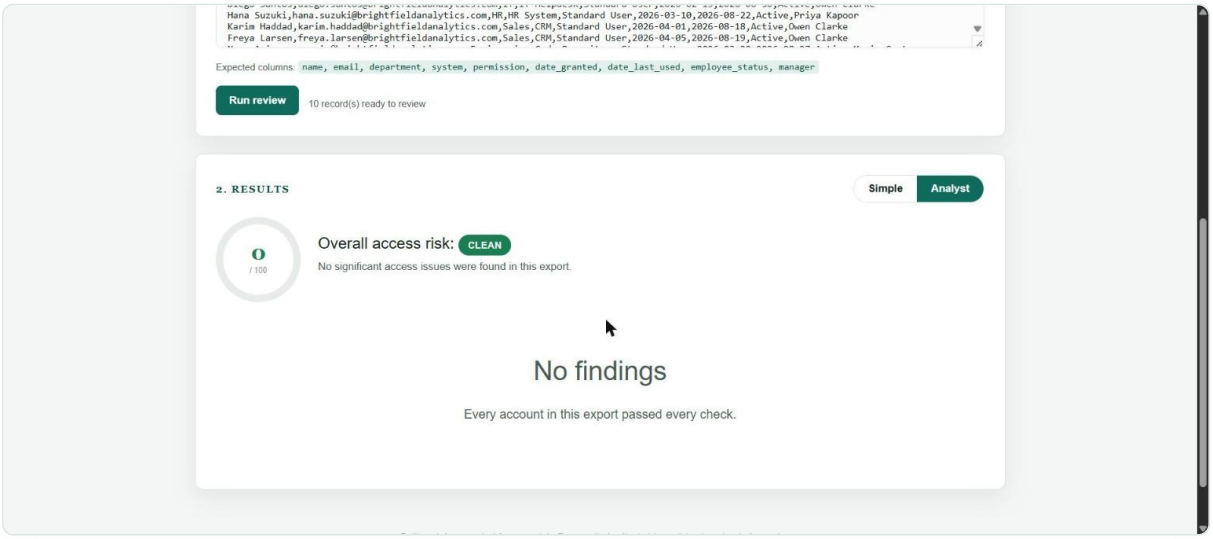
The analyst view with full evidence



The analyst report showing severity, evidence, and recommendations

The same review, one click deeper. Every finding shows its code, its evidence, and a recommended next step, grouped by account and by any company wide issue that is not tied to a single person.

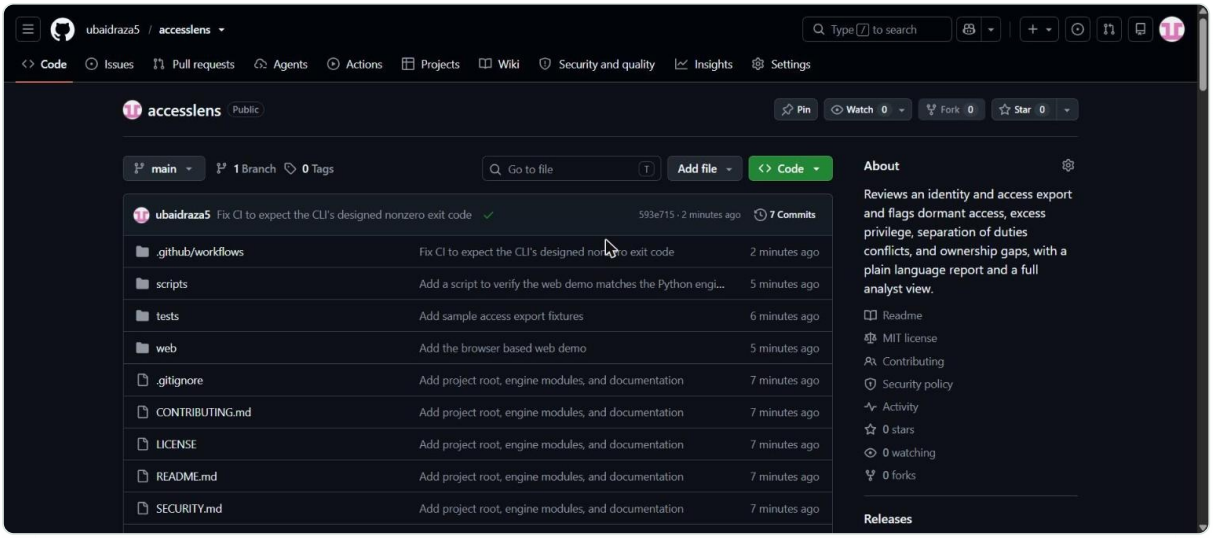
A clean result



A clean company export with no findings

Brightfield Analytics is a second fictional company export with no issues at all, included so a reviewer can see exactly what a well handled access setup looks like, a score of zero and a Clean result.

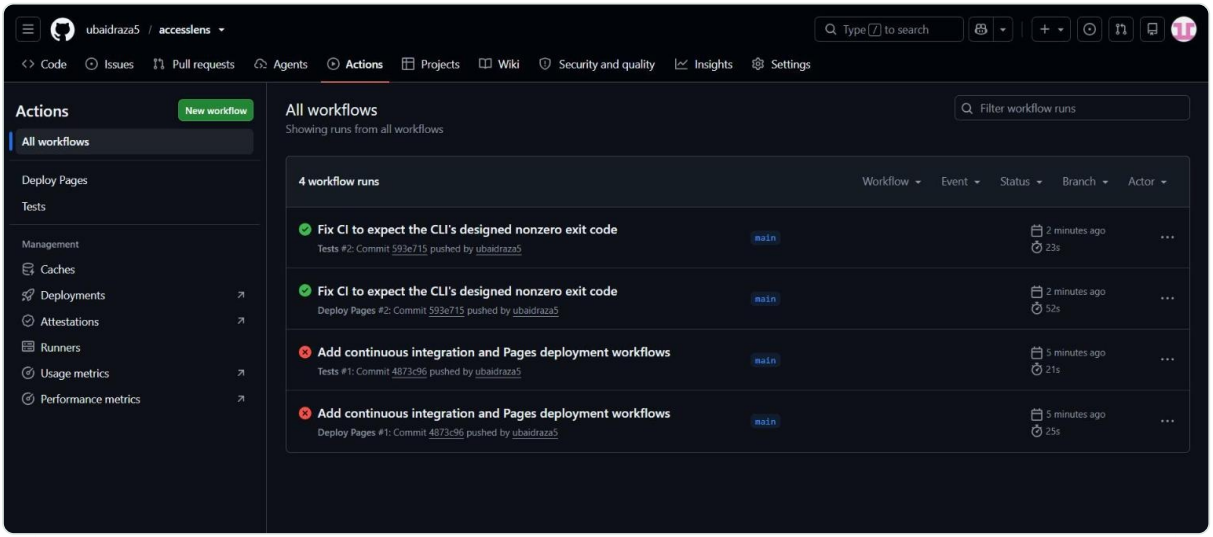
The GitHub repository



The AccessLens repository on GitHub

The full source code, test suite, and documentation, all publicly available and built from scratch.

Continuous integration, all green



GitHub Actions showing the test and deployment workflows passing

Every change runs the pytest suite across four Python versions, checks the browser demo against the Python engine, and only deploys the live site once both checks pass. The earlier failed runs visible here were real issues found and fixed during development, see `DEVELOPMENT_NOTES.md` for the full story of each one.

Usage Guide

This page covers every way to run AccessLens, the command line tool and the browser demo, along with the full list of options and what the output means.

Preparing your access export

Export your access records from whatever system holds them, Active Directory, Okta, Azure AD, a cloud provider console, or a spreadsheet you maintain by hand, and save it as a CSV file with these columns in any order.

```
name, email, department, system, permission, date_granted, date_last_used, employee_status, manager
```

A few notes on the columns.

email should be the same value every time a given person appears, this is how AccessLens tells that two rows belong to the same account.

date_granted and date_last_used should be written as YYYY MM DD, for example 2026 03 15. Day first and month first formats are also accepted. Leave date_last_used blank if the account has never been used since it was granted.

employee_status should read Active or Terminated. Any other value is treated as active, so make sure terminated staff are marked clearly, this is one of the most important signals AccessLens looks for.

manager should be the name of the person accountable for this access. A blank manager is itself flagged, since access nobody owns cannot be reviewed properly.

Running the command line tool

The simplest run reads a file and prints the plain language summary.

```
python main.py my_export.csv
```

Choosing a report style

```
python main.py my_export.csv --mode simple
python main.py my_export.csv --mode analyst
```

Simple mode is the default, and is meant to be read by anyone in the business. Analyst mode prints every finding with its evidence and is meant for a security or audit reviewer who needs to verify the result.

Saving a JSON report

```
python main.py my_export.csv --json report.json
```

This writes the full report, every finding, every account score, and the company wide score, as JSON, which is useful for feeding into another tool or dashboard.

Naming the company in the simple report

```
python main.py my_export.csv --company "Example Holdings"
```

Choosing the reference date

By default AccessLens compares every date against today. To review an export as of a specific date, for example when checking a snapshot taken in the past, pass:

```
python main.py my_export.csv --as-of 2026-06-01
```

Reading the exit code

The command exits 0 when the overall risk level is Clean or Low, and 1 when it is High or Critical. This makes AccessLens easy to drop into a script or a pipeline as a pass or fail gate.

Using the browser demo

Open the live demo at https://ubaidraza5.github.io/accesslens and you will see three ways to provide data: load one of the two built in sample companies, upload your own CSV file, or paste CSV text directly into the box.

Press Run review once you have data in the box. The results panel appears below with an overall score, and a Simple and Analyst toggle at the top right lets you switch between the two views instantly, without rerunning the review.

Nothing you paste or upload in the browser demo is sent to a server, the entire analysis runs using JavaScript already loaded on the page. You can confirm this yourself by opening your browser’s network panel while using it, there is nothing to see there.

Reading a finding

Every finding, in either report style, includes the same underlying information.

A code identifies the type of issue, for example DORMANT_ACCOUNT or SOD_CONFLICT, shown only in analyst mode.

A severity from Info up to Critical shows how serious the issue is.

A message explains the issue in one plain sentence.

Evidence, shown in analyst mode, lists the exact facts behind the finding, dates, systems, and permission names, so a reviewer can check the result without redoing the analysis.

A recommendation suggests the next concrete step to take.

Understanding the score

Each finding carries a weight based on its severity, Critical findings weigh the most, Info findings weigh nothing. An account’s score is the sum of its findings’ weights, capped at 100. The company wide score works the same way across every finding in the export. A single Critical finding also puts a floor under the company level, so one serious issue can never be hidden by a low overall point total.

LEVEL	MEANING
Clean	No issues found
Low	Minor cleanup worth doing, nothing urgent
Medium	Worth reviewing this cycle
High	Needs attention soon
Critical	Needs action right away

Getting help

```
python main.py --help
```

Security

How AccessLens handles your data

The command line tool only ever reads the file path you give it. It does not open a network connection, does not phone home, and does not write anything to disk unless you pass `--json` to ask for a saved report.

The browser demo runs entirely on your own device. The CSV you paste or upload is read by JavaScript already loaded in the page and is never sent to a server, there is no backend for this project at all, the whole analysis, scoring, and report rendering happens in your browser. You can verify this by opening your browser's network panel while using the demo, or by reading the source directly in `web/index.html`.

What kind of data this tool is meant for

AccessLens is built to review identity and access exports, records that typically contain names, email addresses, and the systems and permission levels a person holds. Treat this the same way you would treat any other export from your identity systems, only run it against data you are authorized to review, and only through channels your organization approves of. The live demo is convenient for testing with sample data or your own export on your own machine, but if your organization requires reviews to happen inside a specific approved environment, run the command line tool there instead.

Reporting a security issue

If you find a security problem in this project, please open an issue on GitHub describing the problem. As this is an independent portfolio project rather than a commercial product, there is no formal disclosure program, but any report will be looked at and addressed.

Dependencies

The Python package uses only the standard library, there are no third party runtime dependencies to track for vulnerabilities. The test suite depends on pytest, listed in `requirements-dev.txt`. The browser demo uses no external libraries at all, everything it needs is written into `web/index.html`.

Contributing

This started as a personal portfolio project, but suggestions and fixes are welcome.

Getting set up

```
git clone https://github.com/ubaidraza5/accesslens.git
cd accesslens
pip install -r requirements-dev.txt
pytest
```

All of that should pass with no setup beyond a normal Python install, there are no third party runtime dependencies.

Making a change

Keep each analyzer focused on one review area, the pattern used throughout this project is a function that takes the parsed records and returns a list of Finding objects, nothing more. If you are adding a new kind of check, that usually means a new analyzer module rather than adding logic to an existing one.

If your change affects the scoring or the analyzers, update `web/index.html` to match, the JavaScript engine there is meant to behave identically to the Python one. Running `node scripts/verify_web_demo.js` after a change confirms the two still agree.

Add tests for anything you change. The existing suite is a good template, each analyzer has its own test file under `tests/`, built around small CSV rows constructed with the `record()` helper in `tests/helpers.py`.

Before opening a pull request

```
pytest
node scripts/verify_web_demo.js
```

Both should pass. Please also keep documentation free of unnecessary punctuation and jargon, the goal of this project is that a non specialist can read the plain report and understand exactly what to do next.

Reporting a bug

Open an issue with the CSV row (with any real names replaced) that triggered the unexpected result, and what you expected to see instead. That is normally enough to reproduce and fix the problem quickly.

Development Notes

A record of the real problems I ran into while building AccessLens, and how I solved each one. I am keeping this because these are exactly the kind of issues that come up in a real access review or audit engagement, catching them here was good practice, and it is honest to show the working, not just the finished result.

A single critical issue was not being scored as urgent enough

The scoring model gives every finding a weight based on its severity, and adds them up into a score out of 100 for the whole company. Early on, a company export with exactly one Critical finding, for example one terminated employee still holding admin access, only scored high enough to read as Medium overall, because a single finding’s weight was not large enough on its own to cross the Critical or High point thresholds.

That is the wrong answer for a reviewer. One account left over from a departed employee is a real problem on the day it is found, it should never be filed under Medium just because nothing else is wrong yet.

The fix was to add a severity floor on top of the point total. Any Critical finding now pulls the whole company level up to at least High, and multiple Critical findings, or enough total points, still push it all the way to Critical. The point total and the floor are combined by taking whichever reads more severe, so volume and severity both get a say instead of one hiding the other. I added a dedicated test for this exact case, a single Critical finding, to make sure the floor logic never regresses.

The score looked right but was unreadable in the browser demo

The browser demo shows the company score as a circular gauge, a colored ring with the number in the middle. Once I tested it with a Critical result, the ring filled the whole circle with red, and the number in the middle disappeared, because I had set the number’s text color to the same red as the ring behind it. It was correct data, rendered invisibly.

I fixed this by turning the gauge into a proper donut shape, a colored ring with a plain white circle cut out of the middle, and put the number on that white circle instead of directly on the color. The fix took a small CSS change, wrapping the number in its own inner circle element with a white background, but I only caught it because I actually looked at a screenshot of a Critical result instead of just trusting the numbers.

A mistyped separator silently broke the privilege check in the browser version

The JavaScript version of the privilege check groups people by their department and system, so it can tell if one person stands out compared to their team. To build a unique key for each group, I combined department and system into one string, and meant to put a clear separator between them, but a stray character ended up in the file instead, so two completely different groups, like IT and Console versus ITC and onsole, could end up sharing the same key by accident.

This is the kind of bug that does not throw an error, it just quietly produces a slightly wrong grouping, which is worse than a crash because nothing tells you to go look for it. I caught it by comparing the raw bytes of the file, not just what an editor displayed, since the stray character did not render as anything visible. Once I saw it in a hex dump I replaced it with an explicit, visible separator and re ran the parity check described below to confirm the JavaScript and Python versions still agreed on every test case.

The JavaScript demo and the Python tool could have quietly drifted apart

AccessLens ships two implementations of the same analysis, a Python command line tool and a JavaScript version that runs in the browser demo. Keeping the logic identical by hand, across two languages, is exactly the kind of thing that drifts over time as one side gets a small fix the other does not.

Rather than trust that by eye, I wrote a small Node.js script that pulls the exact script block shipped inside the web demo, runs it against the same fixture files the Python test suite uses, and checks that both sides produce the same score, the same risk level, and the same count of findings at every severity. That script now runs in continuous integration on every change, so the two versions cannot silently disagree without the build failing.

The automated tests failed for a result that was actually correct

The command line tool is designed to exit with a nonzero status when the overall result is High or Critical risk, on purpose, so it can be used as a pass or fail gate in another script or pipeline. When I first wired up continuous integration, the test steps that ran the tool against the sample company, which is deliberately Critical, immediately failed the build, because the automation platform treats any nonzero exit code as a failed step by default.

The tool was behaving exactly as designed, the test around it was wrong. I fixed the affected steps to expect that nonzero exit and treat it as a pass, while still failing the build if the tool ever unexpectedly returned success on a Critical export, which would indicate a real regression in the scoring logic. It is a good example of a test needing to understand what correct behavior actually looks like, rather than assuming success always means a zero exit code.

GitHub Pages did not go live on the first push

After pushing the site for the first time, the live demo returned a site not found page. The repository setting for GitHub Pages defaults to Deploy from a branch, with no branch chosen, which leaves Pages disabled even once a deployment workflow exists and runs successfully. The fix was switching that setting to GitHub Actions as the source, after which the next push deployed normally. I now check that setting immediately after creating a new repository, rather than after wondering why the site is not loading.

What this adds up to

None of these were exotic problems, a scoring edge case, a rendering bug only visible with the right data, a typo that produced no error message, a test that assumed the wrong thing, and a repository setting that is easy to miss. That is a fairly accurate picture of what real review and audit work looks like too, most issues are not dramatic, they are small, specific, and only found by actually checking the output rather than assuming it is right.